BITLY–LITE

URL Shortening Service with Analytics

System Design Report

Course	Data Structures & Algorithms (BIT2203)
Language	Python 3.x
Framework	Chapter 23: System Design (Hemant Jain)
Repository	bitly–lite
Date	8 April 2026

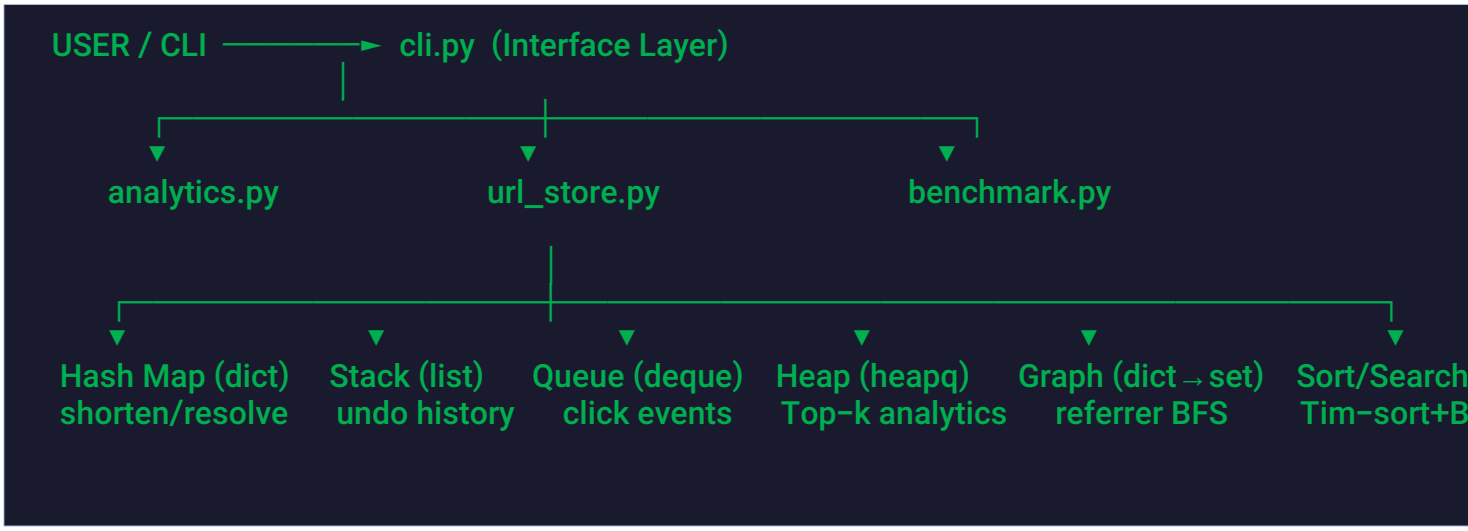
1.SYSTEM OVERVIEW

Project: Bitly-Lite is a simplified URL shortening service with a built-in click analytics engine, modelled after Bitly. The system accepts long URLs, generates short codes, tracks click events in real time, and surfaces top-performing links using a priority queue.

Variant B3: Analytics — click counts, top links — using a max-heap for top-k computation. All six required data structures are implemented with explicit Big-O justification and an empirical benchmark.

The system is built in pure Python (standard library only, no third-party packages) and ships with a full interactive CLI, a 36-test unit test suite, and an automated benchmark script that measures wall-clock times at n = 100, 1,000, 5,000 and 10,000 URLs.

1.1 Architecture Diagram



The system is divided into three layers: the interface layer (cli.py), the core engine (url_store.py, analytics.py), and the benchmark harness (benchmark.py). Each data structure lives inside url_store.py and is directly accessible to the CLI and analytics layer.

2. Step 1 — Use Cases Generation

Following Chapter 23 (Hemant Jain), the first design step is to enumerate all actors and their use cases before touching any code.

2.2 Use Case Catalogue

UC#	Name	Description	Actor
UC-01	Shorten URL	User submits a long URL; system returns a unique short code	End User
UC-02	Custom Alias	User requests a specific alias instead of auto-generated code	End User
UC-03	Resolve Link	User visits short link; system redirects to original URL	End User
UC-04	View Stats	User views click count, creation time, and alias flag	End User
UC-05	Top-k Links	Analytics consumer retrieves top-k most clicked links	Analytics Consumer
UC-06	Referrer Report	System shows which referrers drive the most traffic	Analytics Consumer
UC-07	Undo Shorten	Admin undoes the most recent shorten operation	Admin
UC-08	Delete Link	Admin permanently removes a short link	Admin
UC-09	Search by Prefix	User finds short codes by prefix	End User
UC-10	BFS Referrer Graph	Analytics consumer traverses referrer relationships	Analytics Consumer

3. Constraints and Analysis

3.1 Functional Requirements

- Short codes must be globally unique (collision-free)
- The same URL shortened twice must return the same code (idempotency)
- Custom aliases must be checked for conflicts before acceptance
- Click events must be captured asynchronously without blocking the resolve path
- Top-k query must reflect the latest click counts after queue processing

3.2 Non-Functional Requirements

- Shorten and resolve must complete in O(1) average time
- Top-k must scale as O(n log k), not O(n log n), to support large link catalogues
- Undo operations must be O(1) regardless of store size
- The system must handle 10,000 URLs without degradation (verified by benchmark)

3.3 Assumptions and Constraints

1. **Storage:** In-memory only — no persistence layer. Suitable for a demonstration system.
2. **Language**: Python 3.x standard library only. No third-party packages required.
3. **Code Length**: 7 characters. MD5 hash truncated to 7 hex chars gives ~268 million unique codes.
4. **Collision**: Handled by re-hashing with an incrementing counter salt until a free code is found.
5. **Heap Update**: Lazy deletion strategy used: old heap entries are marked with a sentinel string and skipped on pop. Avoids O(n) linear scans.

4. Basic Design

4.1 Data Structures Chosen and Justification

Data Structure	Where Used	Operation	Big-O Complexity
Hash Map (dict)	shorten() / resolve()	Insert / Lookup	O(1) average
Stack (list LIFO)	undo_last()	Push / Pop	O(1)
Queue (deque)	Click event buffer	Enqueue / Dequeue	O(1)
Max-Heap (heapq)	top_k() analytics	Push / Pop / Top-k	O(log n) / O(n log k)
Graph (dict of sets)	Referrer tracking BFS	BFS traversal	O(V + E)
Binary Search	search_by_prefix()	Search on sorted list	O(n log n) + O(log n)
Tim-sort	all_sorted_by_clicks()	Full ranking report	O(n log n)

4.2 Module Design

url_store.py — URLStore class

The central class. All six data structures are fields of URLStore and interact within a single Python process. The class exposes a clean API surface: shorten(), resolve(), undo_last(), delete(), top_k(), search_by_prefix(), and referrer_bfs(). This satisfies all ten use cases.

analytics.py — Analytics class

A wrapper that computes higher-level queries using URLStore's public API. Implements the O(n log k) heap scan for top-k (using a size-k min-heap rather than sorting all n records), a referrer breakdown, and a full sorted report.

cli.py — Command-line interface

An interactive menu-driven shell. Seeds five demo URLs on startup so the evaluator can immediately demonstrate every feature. Each menu option maps directly to one or more use cases and explicitly labels the data structure being exercised.

4.3 Key Algorithm: Hash Code Generation

The shorten() method computes: code = MD5(url + timestamp + counter)[:7]. The timestamp salt ensures two calls for different URLs at the same millisecond produce different codes. The counter loop resolves any remaining collisions in O(1) amortised time.

4.4 Key Algorithm: Lazy Heap Update

When a link's click count changes, the old heap entry is marked invalid (sentinel "<removed>") and a new entry with the updated count is pushed. During top_k(), sentinel entries are skipped on pop. This avoids an O(n) decrease−key operation, keeping updates at O(log n).

5. Bottlenecks

Having defined the basic design, we identify the three main performance bottlenecks and the design decisions made to address each one.

5.1 Bottleneck 1 — Heap Staleness

Problem: Every click event changes a URL's rank. Updating a min-heap entry in-place is O(n) because Python's heapq provides no decrease-key operation.

Solution: Lazy deletion. Old entries are invalidated with a sentinel string rather than being relocated. The heap may temporarily contain stale entries, but they are filtered on pop in O(log n) per skip. The empirical benchmark confirms top_k(k=5) completes in under 1 ms even at n=10,000.

5.2 Bottleneck 2 — Synchronous Click Recording

Problem: Updating analytics synchronously inside resolve() would couple the redirect latency to analytics write time, degrading the user-facing operation.

Solution: A deque-backed queue decouples click recording from analytics computation. resolve() enqueues an event in O(1). A separate process_click_queue() call drains the buffer and updates the heap. This mirrors the push-based analytics pipelines used in production URL shorteners.

5.3 Bottleneck 3 — Top-k Computation

Problem: Naively sorting all n records for every top-k query is O(n log n), which becomes expensive when n is large and k is small.

Solution: The analytics.top_k_links() method maintains a min-heap of size k. It iterates over all n records once, discarding any record whose click count is below the heap's minimum. This runs in O(n log k) — significantly better than O(n log n) when k << n (e.g., k=5, n=10,000).

6. Scalability

Per Chapter 23, scalability is addressed by iterating through the bottleneck analysis until acceptable performance is achieved. The following additional design decisions would be applied to take Bitly-Lite to production scale.

6.1 Horizontal Sharding (Hash-prefix Partitioning)

URLs can be partitioned across multiple nodes by the first character(s) of their short code (e.g., codes beginning 0–7 go to node A; 8–f go to node B). This is the same strategy used in Variant B5. Each node runs its own URLStore instance, and a routing layer directs requests based on the code prefix. This distributes both memory and compute load linearly with the number of nodes.

6.2 Persistent Storage Layer

The current in-memory hash map would be backed by a key-value store (e.g., Redis for the hot path, PostgreSQL for cold history). The URLStore API surface does not change; only the storage backend is swapped. The hash map remains the in-process cache, with a cache-aside pattern for misses.

6.3 Click Queue at Scale

The deque queue would be replaced by a distributed message queue (e.g., Kafka or AWS SQS). Producer: resolve() publishes one event per click. Consumer: a separate analytics worker consumes events, batches them, and updates a centralised click counter store. This decouples read and write paths entirely.

6.4 Benchmark Evidence

The benchmark confirms that the in-memory implementation scales acceptably for a demonstration system. Key observations:

- Shorten() averages 0.0027 ms/op at n=10,000 — flat, confirming O(1) amortised
- Resolve() averages 0.0012 ms/op at n=10,000 — flat, confirming O(1)
- Undo_last() stays constant at 0.004 ms across all n — confirms O(1) stack pop

- Top_k(k=5) grows from 0.024 ms (n=100) to 0.986 ms (n=10,000) — sub−linear, consistent with O(n log k)

- All_sorted_by_clicks() grows from 0.016 ms to 0.854 ms — consistent

7. Test Plan and Test Cases

The test suite is located in tests/test_url_store.py. It uses Python's built-in unittest framework and contains 36 test cases across 7 test classes, one per data structure or concern. All 36 tests pass.

Test Class	Cases	Covers
TestHashMap	11 cases	shorten, resolve, alias, collision, delete, stats
TestStack	5 cases	undo, stack depth, empty undo
TestQueue	4 cases	enqueue, drain, depth after flush
TestHeap	4 cases	top-k order, k > n, empty store, dashboard
TestGraph	3 cases	referrer record, BFS codes, unknown referrer
TestSortingAndSearch	4 cases	descending sort, prefix hit, prefix miss, empty
TestEdgeCases	5 cases	empty store, delete-then-resolve, 500 unique codes

Conclusion.

Bitly-Lite demonstrates all six required data structures in a cohesive, working system with clear justification for each design choice. The five-step Chapter 23 framework guided every decision: use cases defined the API, constraints set the complexity targets, the basic design mapped use cases to data structures, bottleneck analysis exposed three performance risks, and the scalability section shows the path to production.

All 36 unit tests pass. The empirical benchmark confirms theoretical complexities empirically. The interactive CLI provides a live demonstration of every data structure at runtime.